Notas de presentación

GA-TL-01 · Taller de modelado de comunicación entre procesos distribuidos
Escenario 4: Cooperativa Ahorro Futuro · Grupo CMSZ

C: Cedeño Intriago Jilmar Jesús
M: Mendoza Bermello Angello Agustín
S: Sánchez Vera Darwin Joel
Z: Zambrano Triviño Leonel Leonidas

Aplicaciones Distribuidas (ISR-701) · Entrega: miércoles 11/11/2026

1. Escenario y requisitos

Cooperativa Ahorro Futuro es una cooperativa de ahorro y crédito con agencias en Los Ríos y Guayas. Sus socios usan banca web, cajeros automáticos propios y ventanillas. Los saldos viven en un core bancario heredado que no se puede reemplazar a corto plazo. En los días de pago las transferencias se demoran, hay débitos duplicados cuando el socio pulsa dos veces «enviar» y algunos cajeros entregan dinero sin que el débito quede registrado.

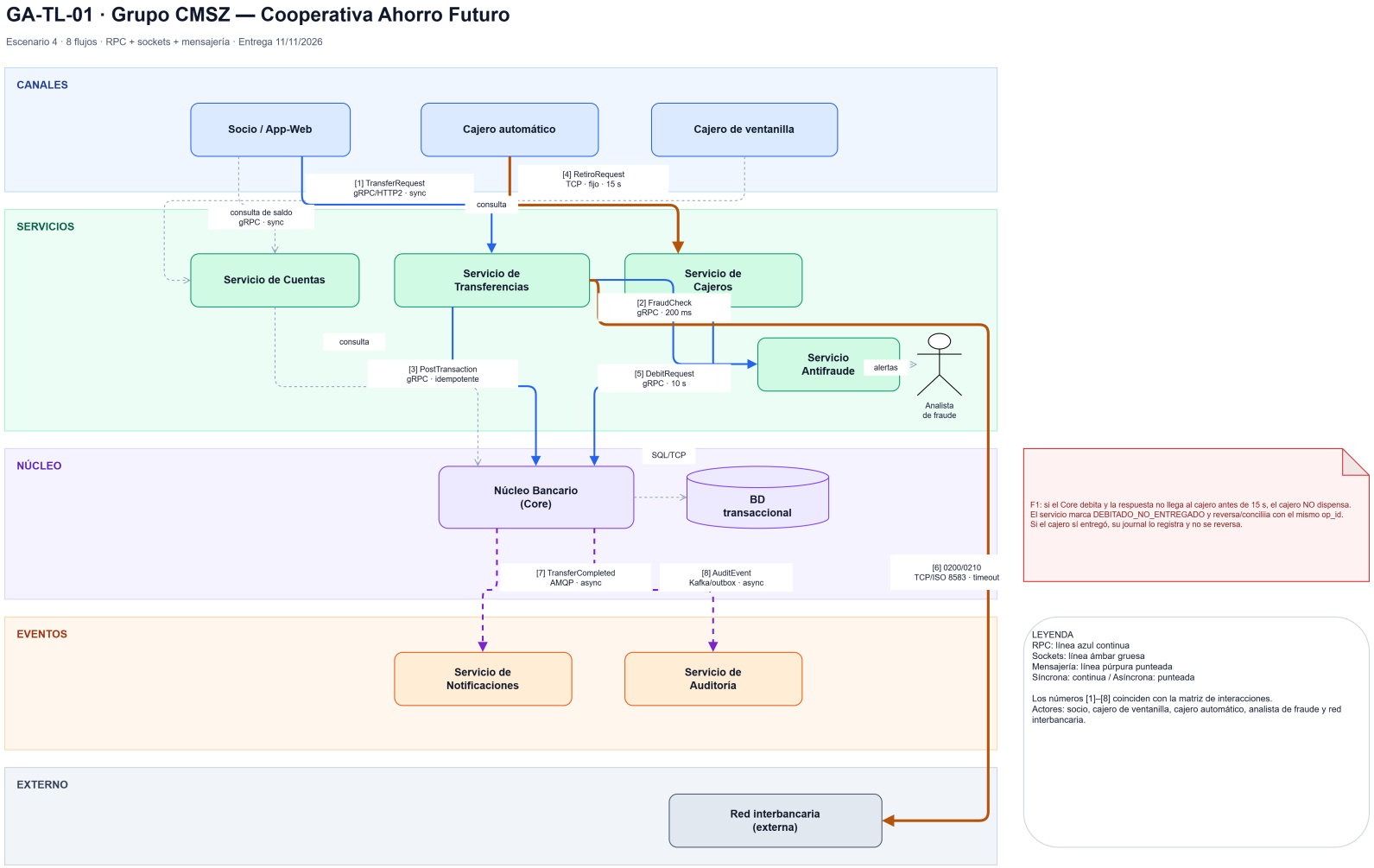
Actores: socio, cajero de ventanilla, cajero automático, analista de fraude y red interbancaria.

Operaciones modeladas: (a) notificar el movimiento al socio y registrarlo en auditoría; (b) transferir entre cuentas de la cooperativa; (c) transferir a otro banco por la red interbancaria; (d) retirar efectivo en un cajero automático; (e) consultar el saldo.

Requisitos R1–R6 y cómo los cumple el diseño

Req.	Qué exige	Cómo se cumple	Flujos
R1	300 transacciones por segundo en días de pago; el 95 % en menos de 2 s	gRPC/HTTP2, pooling, cola RabbitMQ que absorbe picos y prueba de carga	1, 7
R2	El saldo se confirma en el core; nunca se debita dos veces la misma operación	Idempotency key + restricción única + estado de operación	3, 5
R3	Antifraude responde en menos de 200 ms; el equipo define qué hacer si no responde	Deadline de 200 ms + circuit breaker + política de contingencia	2
R4	Cajeros con mensajes de formato fijo y timeout máximo de 15 s; sin respuesta no entregan dinero	Sockets TCP, framing, formato fijo y timeout de 15 s	4
R5	Toda operación queda en auditoría, aunque el servicio de auditoría esté caído	Outbox transaccional + Kafka durable + reintentos	8
R6	Disponibilidad del 99,95 % para transferencias y retiros	Réplicas, health checks, reintentos con backoff y degradación graceful	2, 6, 7, 8

2. Diagrama de comunicación



Archivo editable: `diagrama/sistema-bancario.drawio`. Los números [1]–[8] de las flechas coinciden con la matriz de interacciones de la sección 3.

3. Matriz de interacciones

#	Origen	Destino	Tipo	Sync/Async	Protocolo / middleware	Justificación y requisito	Dueño
1	App/Web	Servicio de Transferencias	RPC	Síncrona	gRPC-Web sobre HTTP/2	El socio requiere respuesta inmediata; contrato tipado y multiplexación para 300 TPS y p95 menor de 2 s (R1). La solicitud conserva una clave de idempotencia (R2).	C
2	Servicio de Transferencias	Servicio Antifraude	RPC	Síncrona; deadline de 200 ms	gRPC sobre HTTP/2, deadline y circuit breaker	El análisis debe anteceder al débito (R3). Ante una caída se aplica contingencia diferenciada, sin detener todas las operaciones (R6).	C
3	Servicio de Transferencias	Core bancario	RPC	Síncrona	gRPC sobre HTTP/2, adaptador del core, idempotency key y pool de conexiones	Solo el core confirma saldos oficiales; la misma operación no genera doble débito (R2).	S
4	Cajero automático	Servicio de Cajeros	Sockets	Síncrona; timeout máximo de 15 s	TCP, mensajes de formato fijo, framing length-prefix; mTLS opcional	El dispositivo heredado usa formato fijo y no entrega efectivo sin confirmación (R4).	M
5	Servicio de Cajeros	Core bancario	RPC	Síncrona; deadline de 10 s	gRPC sobre HTTP/2 con idempotencia	El adaptador consulta al sistema de registro y confirma el débito antes de autorizar la entrega de efectivo (R2, R4).	M
6	Servicio de Transferencias	Red interbancaria	Sockets	Síncrona; timeout controlado	TCP y mensajes de formato fijo (supuesto: ISO 8583)	Se integra una red externa heredada, con estados de proceso, conciliación y compensación ante incertidumbre (R6).	S
7	Transferencias / Core (publica el relay del outbox)	Servicio de Notificaciones	Mensajería	Asíncrona	AMQP 0-9-1 con RabbitMQ; reintentos y DLQ	El socio no espera el envío de SMS o correo; la cola absorbe picos y permite reintentos (R1, R6).	Z
8	Core / Transferencias	Servicio de Auditoría	Mensajería	Asíncrona	Kafka y outbox transaccional; clave de partición `cuenta`	El evento persiste aunque auditoría no esté disponible y se entrega después (R5).	Z

4. Flujos por responsable

Cada integrante modeló y justificó dos flujos. En cada ficha se indica el tipo de comunicación, si es síncrona o asíncrona, el protocolo o middleware, el mensaje y los requisitos que atiende.

4.1. Canal del socio y antifraude (Flujos 1 y 2)

Flujo 1 — App/Web → Servicio de Transferencias

Tipo	RPC
Sync/Async	Síncrona
Protocolo	gRPC-Web sobre HTTP/2
Mensaje	`TransferRequest{idempotency_key, cuenta_origen, cuenta_destino, monto, moneda}`
Requisitos	R1, R2
Dueño	C

Por qué. El socio necesita conocer el resultado al confirmar la transferencia. El contrato tipado de gRPC define el mensaje y su respuesta, y HTTP/2 permite multiplexar solicitudes para la meta de 300 TPS con p95 menor de 2 s (R1). La misma `idempotency_key` se conserva en los reintentos (R2). Para un navegador, gRPC-Web puede requerir un proxy compatible; es un supuesto de despliegue, no una interacción nueva.

Si falla. Si se interrumpe la comunicación o vence el tiempo de espera, la interfaz no asume éxito ni fracaso: muestra `PENDIENTE` y cualquier consulta o reintento usa la misma `idempotency_key`; nunca se generan dos débitos.

Flujo 2 — Servicio de Transferencias → Servicio Antifraude

Tipo	RPC
Sync/Async	Síncrona; deadline de 200 ms
Protocolo	gRPC sobre HTTP/2, deadline de 200 ms y circuit breaker
Mensaje	`FraudCheckRequest{op_id, cuenta, monto, beneficiario, canal}` → `APROBAR | RECHAZAR | REVISAR`
Requisitos	R3, R6
Dueño	C

Por qué. La revisión de riesgo debe ocurrir antes del débito en el core. R3 exige respuesta en menos de 200 ms, y una llamada gRPC síncrona permite fijar ese deadline. Como la dependencia síncrona puede afectar la disponibilidad, un circuit breaker limita las llamadas a un servicio inestable y activa la contingencia (R6).

Si falla (F2). Ni se detienen todas las transferencias ni se aprueban sin revisión: las de monto bajo sin alertas continúan marcadas para revisión posterior; las de monto alto o riesgo elevado quedan en `PENDIENTE_REVISION`. Toda decisión de contingencia queda registrada para auditoría. El umbral entre monto bajo y alto no está definido en la actividad y corresponde a una política institucional.

4.2. Cajero automático y sockets (Flujos 4 y 5)

Flujo 4 — Cajero automático → Servicio de Cajeros

Tipo	Sockets TCP
Sync/Async	Síncrona; timeout de 15 s
Protocolo	TCP con framing length-prefix, formato fijo y mTLS opcional

Mensaje	`RetiroRequest`, formato fijo:
	`[len][ver][tipo][tarjeta][monto]`
	`[cajero_id][op_id][crc]`

Requisitos	R4, R6
Dueño	M

Por qué. R4 exige formato fijo y timeout de 15 s. El cajero es legacy; los sockets TCP permiten controlar el framing, la conexión persistente y el timeout. No se dispensa sin respuesta.

Si falla. Sin respuesta en 15 s el cajero no dispensa; la operación queda `EN_PROCESO` y se consulta o revierte con el `op_id`; luego se concilia. Estados de la operación: `INICIADA`, `DEBITADA`, `ENTREGADA`, `DEBITADO_NO_ENTREGADO`, `REVERSADA` y `EN_PROCESO`.

Flujo 5 — Servicio de Cajeros → Core

Tipo	RPC (gRPC)
Sync/Async	Síncrona; deadline de 10 s
Protocolo	gRPC/HTTP2 con idempotencia y deadline de 10 s
Mensaje	`DebitRequest{op_id, tarjeta, monto, cajero_id}`
Requisitos	R2, R4
Dueño	M

Por qué. El servicio de cajeros actúa como adaptador: recibe el formato fijo por socket y llama al core por gRPC. El débito se confirma antes de que el cajero entregue dinero, y la idempotencia protege los reintentos. El deadline de 10 s deja margen dentro de los 15 s del cajero.

Si falla. Si el core no responde en 10 s, el servicio responde `NO_DISPENSAR`: el cajero no entrega dinero, la operación se marca `EN_PROCESO` y se revierte o concilia.

4.3. Núcleo y red interbancaria (Flujos 3 y 6)

El core es el sistema de registro: ninguna transferencia se da por hecha hasta que el core la confirma, y la misma operación nunca se debita dos veces (R2). El `op_id` identifica la operación en todos los flujos que la tocan, incluidos los reintentos, la reversa y la conciliación.

Flujo 3 — Servicio de Transferencias → Core

Tipo	RPC
Sync/Async	Síncrona
Protocolo	gRPC/HTTP2 hacia el adaptador gRPC del core, con idempotency key y pool de conexiones
Mensaje	`PostTransaction{op_id, débito, crédito, monto}`
Requisitos	R2, R1
Dueño	S

Por qué. El saldo oficial vive en el core. Transferencias necesita una confirmación síncrona antes de responder al socio, y el débito y el crédito deben quedar en una sola transacción: consistencia fuerte.

gRPC/HTTP2 aporta contrato tipado y deadline; el pool de conexiones reutiliza las conexiones con el adaptador para el pico de 300 TPS (R1). La restricción única sobre `op_id` evita el doble débito (R2).

Secuencia.

1. Con la aprobación de antifraude (flujo 2), Transferencias envía `PostTransaction` con `op_id`, las cuentas de débito y crédito y el monto.
2. El core valida la restricción única de `op_id`.
3. Si el `op_id` es nuevo, debita y acredita en una sola transacción; en esa misma transacción quedan en el outbox los eventos de la operación (flujos 7 y 8).
4. Si el `op_id` ya existe, devuelve el resultado original sin debitar otra vez.
5. El core responde OK o error; Transferencias actualiza el estado y responde al canal.

Si falla. Ante un timeout o una respuesta perdida, la operación queda `PENDIENTE`: Transferencias consulta al core por `op_id` y, si reintenta, lo hace con el mismo `op_id`, de modo que el core devuelve el resultado original. Si el estado sigue incierto, la conciliación lo resuelve con el mismo identificador.

Trade-off. Ganamos consistencia fuerte y débito único. Pagamos que la transferencia depende de que el core responda y que un timeout deja un estado incierto que obliga a consultar y conciliar.

Flujo 6 — Servicio de Transferencias → Red interbancaria

Tipo	Sockets
Sync/Async	Síncrona, con timeout
Protocolo	TCP con mensajes de formato fijo (supuesto: ISO 8583)
Mensaje	0200/0210 · `op_id`, monto, beneficiario, banco destino
Requisitos	R6, R2
Dueño	S

Por qué. La red interbancaria es externa y legacy: no se controla su latencia ni se puede cambiar su protocolo, así que no se usa gRPC sino el formato fijo que la red expone (supuesto: ISO 8583 sobre TCP; 0200 es la solicitud y 0210 la respuesta). Un timeout corto evita que Transferencias quede bloqueado por una red lenta: la operación queda `EN_PROCESO` y se resuelve después (R6). Si la red rechaza, el débito se compensa (R2).

Secuencia.

1. El débito de la cuenta del socio ya quedó registrado en el core por el flujo 3; por eso un rechazo se corrige con una reversa.
2. Transferencias envía 0200 con `op_id`, monto, beneficiario y banco destino.
3. La red responde 0210 con aprobación o rechazo.
4. Si aprueba, se confirma la operación y se notifica al socio.
5. Si rechaza, se reversa el débito con el mismo `op_id`.
6. Si no responde dentro del timeout, queda `EN_PROCESO`, se reintenta con el mismo `op_id` y se concilia.

Trade-off. Ganamos integración con un sistema que no controlamos sin bloquear el servicio. Pagamos el manejo de framing y formato fijo, un estado intermedio visible para el socio y un proceso de conciliación.

4.4. Eventos: notificaciones y auditoría (Flujos 7 y 8)

Notificaciones y auditoría son procesos de eventos: asíncronos, toleran reintentos y no bloquean la operación principal. Ambos parten de una misma fuente, la tabla *outbox*, donde cada operación deja su evento dentro de la misma transacción que la registra. Un relay lee el outbox y publica en dos middleware distintos, porque resuelven problemas distintos: RabbitMQ como cola de trabajo con reintentos y DLQ para avisar al socio, y Kafka como registro durable y re-legible para la auditoría.

Flujo 7 — Transferencias / Core → Notificaciones

Tipo	Mensajería
Sync/Async	Asíncrona
Middleware	AMQP 0-9-1 con RabbitMQ; reintentos y DLQ
Publica	Relay del outbox (carril Núcleo)
Mensaje	`MovimientoRegistrado{op_id, cuenta, monto, canal, tipo}`
Requisitos	R1, R6
Dueño	Z

Por qué. El socio no necesita esperar a que se envíe el SMS, el correo o la push. RabbitMQ desacopla el pico de 300 TPS de los días de pago (R1), permite reintentos y envía a DLQ los mensajes que no se pueden entregar. Publicar desde el outbox, y no desde Transferencias, evita perder avisos si un proceso cae justo después de que el core confirma, y cubre todo movimiento que pasa por el core: transferencias, retiros en cajero y reversas (operación a).

Secuencia.

1. La operación y su evento quedan en el outbox, en la misma transacción.
2. El relay publica `MovimientoRegistrado` en el exchange de RabbitMQ.
3. RabbitMQ enruta el mensaje a la cola de notificaciones.
4. El consumidor de notificaciones envía SMS, correo o push.
5. Si el envío falla, se reintenta; agotados los reintentos, el mensaje pasa a la DLQ.
6. La operación nunca espera la notificación.

Si falla. Si notificaciones cae, el mensaje queda en la cola (durable, con mensajes persistentes). Si RabbitMQ no está disponible, el evento espera en el outbox y el relay reintenta. En ningún caso se detiene la operación (R6).

Trade-off. Ganamos desacoplamiento, absorción de picos y avisos que no se pierden. Pagamos operar un broker, aceptar que la notificación llega con retraso y que RabbitMQ no garantiza orden entre avisos; como cada aviso trae su monto y su tipo, el desorden no confunde al socio.

Flujo 8 — Core / Transferencias → Auditoría

Tipo	Mensajería
Sync/Async	Asíncrona
Middleware	Kafka con transactional outbox
Mensaje	`AuditEvent{op_id, tipo, actor, timestamp, payload_hash}`
Clave de partición	`cuenta`
Requisitos	R5 (y R6)
Dueño	Z

Por qué. R5 exige que toda operación quede auditada aunque el servicio de auditoría esté caído. Por eso el evento no se envía directo al servicio: se guarda en el outbox dentro de la misma transacción que registra la operación. Si la operación se confirma, su evento también; si se revierte, tampoco queda el evento. Hay dos outbox: el de la BD transaccional del Núcleo (operaciones que pasan por el core) y el de Transferencias (decisiones de antifraude que no llegan al core, como `PENDIENTE_REVISION` en F2). Las consultas de saldo, que no escriben nada, las publica el servicio de Cuentas directo a Kafka.

Secuencia.

1. Se escriben la operación y el `AuditEvent` en la misma transacción (outbox).
2. El relay lee el outbox y publica el evento en Kafka, con la cuenta como clave de partición.
3. El consumidor de auditoría guarda el evento de forma idempotente, usando `(op_id, tipo)` para descartar repetidos.
4. Si auditoría está caída, Kafka retiene el evento y el consumidor continúa desde su último offset al volver.
5. Si Kafka no está disponible, el outbox conserva el evento y el relay reintenta.

Si falla. Outbox persistente; relay con reintentos; consumidor idempotente. Ningún fallo de Kafka o de auditoría bloquea la operación ni pierde el evento.

Por qué `(op_id, tipo)` y no solo `op_id`. Una misma operación puede generar varios eventos. En F1, la operación genera `DEBITO` y luego `REVERSA`, ambos con el mismo `op_id`. Deduplicar solo por `op_id` descartaría la reversa y la auditoría quedaría incompleta.

Por qué partición por cuenta. Kafka solo garantiza orden dentro de una partición. Con la cuenta como clave, los eventos de un mismo socio quedan en orden (débito antes que reversa) y las cuentas distintas se procesan en paralelo (R1).

Trade-off. Ganamos la garantía de R5 sin agregar una dependencia síncrona al camino del dinero: la escritura va a la misma base de datos que ya participa en la operación. Pagamos un componente más (el relay), entrega *al menos una vez* (por eso el consumidor es idempotente) y un retraso entre la operación y su registro en auditoría.

Aporte a R6 (99,95 %)

Los flujos 7 y 8 están fuera del camino crítico de transferencias y retiros: la caída de notificaciones, de auditoría o de los brokers no detiene ninguna operación, porque el evento ya quedó en el outbox. Se apoyan en réplicas, health checks y reintentos con backoff, como indica la matriz R1–R6. Los límites concretos (tamaño de cola, número de reintentos) se definen en la práctica con la prueba de carga.

Supuestos de los flujos 7 y 8

- La tabla outbox está en la BD transaccional del Núcleo; el adaptador del core registra la operación y el evento en la misma transacción de esa base. No se modifica el core heredado.
- R5 cubre todas las operaciones, incluidas las consultas de saldo. Una consulta no modifica nada, así que no hay transacción que proteger: el servicio de Cuentas publica su `AuditEvent` (tipo `CONSULTA`) directo a Kafka, sin outbox, y reintenta sin bloquear la consulta si Kafka no responde.

Preguntas recibidas (Flujos 7–8)

-

5. Consistencia: idempotencia, outbox y compensación (R2 y R5)

Tres mecanismos sostienen las dos garantías del diseño: *nunca debitar dos veces* (R2) y *toda operación queda auditada* (R5).

Mecanismo	Dónde	Qué garantiza	Flujos
Idempotencia	`idempotency_key` desde el canal; restricción única de `op_id` y estado de la operación en el core	Un reintento o un doble clic devuelve el resultado original, no un segundo débito (R2, F3)	1, 3, 5
Outbox transaccional	Tabla outbox en la BD transaccional del Núcleo y en Transferencias	El evento se guarda en la misma transacción que la operación; si la operación se confirma, su evento también, aunque auditoría esté caída (R5)	7, 8
Compensación y conciliación	Reversa con el mismo `op_id`; conciliación cada minuto	Un débito que no termina en transferencia o en entrega de efectivo se revierte; un estado incierto se resuelve sin duplicar efectos (R2, R6)	4, 5, 6

El `op_id` es el hilo común. El mismo identificador viaja en el débito, en los reintentos, en la reversa, en la conciliación y en los eventos. Por eso un reintento se reconoce como la misma operación, y la auditoría deduplica por `(op_id, tipo)` para conservar a la vez el `DEBITO` y la `REVERSA` de una misma operación.

Qué ganamos y qué pagamos. Ganamos que ningún fallo de red provoque un doble débito ni una operación sin auditar. Pagamos estados intermedios (`PENDIENTE`, `EN_PROCESO`), un proceso de conciliación y un retraso entre la operación y su registro en auditoría.

6. Situaciones de fallo F1–F3

F1 — El core debita, pero la respuesta no llega al cajero antes de 15 s (M). El cajero no dispensa sin confirmación. El servicio de cajeros marca `DEBITADO_NO_ENTREGADO` y reversa con el mismo `op_id`; si la reversa falla, la conciliación reintenta. Si el cajero sí entregó, su journal lo registra y no se reversa. El socio no pierde dinero y no hay doble débito.

F2 — Antifraude caído o lento en un día de pago (C). Se abre el circuit breaker y se aplica contingencia: montos bajos sin alertas se aprueban y quedan para revisión posterior; montos altos o de riesgo alto quedan en `PENDIENTE_REVISION`. Nunca se aprueba todo en silencio ni se bloquean todas las transferencias. La decisión se registra para auditoría.

F3 — Dos solicitudes idénticas por doble clic (C, S). La interfaz desactiva el botón, pero la garantía real es la `idempotency_key` por intención más la restricción única en el core. La solicitud duplicada devuelve el mismo resultado, no un segundo débito.

7. Supuestos del equipo

- El core heredado expone un adaptador gRPC; el equipo no lo implementa, solo lo modela.
- La red interbancaria usa mensajes de formato fijo (supuesto: ISO 8583 sobre TCP).
- El cajero automático es legacy y solo habla TCP con formato fijo.
- El timeout del cajero es 15 s; el del core desde el servicio de cajeros es 10 s para dejar margen.
- La conciliación corre cada minuto y usa el mismo `op_id`.

- mTLS/TLS y la autenticación de servicios son parte del diseño, aunque no se detallen en el diagrama.
- Para navegadores, gRPC-Web puede requerir un proxy o gateway compatible; se considera parte de la frontera de entrada y no cambia el flujo 1.
- La tabla outbox está en la BD transaccional del Núcleo. El adaptador del core registra la operación y el evento en la misma transacción de esa base; así no se toca el core heredado.
- R5 cubre todas las operaciones, incluidas las consultas de saldo. Una consulta no modifica nada, así que no hay transacción que proteger: el servicio de Cuentas publica su `AuditEvent` (tipo `CONSULTA`) directo a Kafka, sin outbox, y reintenta sin bloquear la consulta.
- La red interbancaria reconoce un reintento con el mismo `op_id` como la misma operación. El valor exacto de su timeout se fija en la práctica.

8. Guion de la exposición (5 minutos)

Tramo	**Quién**	**Contenido**
0:00–0:30	C	Escenario y problema: core heredado, 300 TPS en días de pago, débitos duplicados y cajeros que entregan sin registrar.
0:30–1:45	C	Flujos 1 y 2: app → transferencias por gRPC; transferencias → antifraude con deadline de 200 ms y política F2.
1:45–3:00	M	Flujos 4 y 5: TCP, formato fijo y timeout de 15 s; el adaptador confirma con el core por gRPC. F1.
3:00–4:15	S	Flujos 3 y 6: idempotencia en el core y compensación en la red externa. R2 y R5.
4:15–4:50	Z	Flujos 7 y 8: RabbitMQ para notificaciones y Kafka con outbox para auditoría; la operación no depende de ellos. R6.
4:50–5:00	Todos	Cierre: «Sockets para el cajero, gRPC para el dinero, mensajería para eventos».

9. Registro de preguntas críticas

Preguntas recibidas de otros equipos

N.º	Equipo	Pregunta (tal como se formuló)	Responde	Respuesta dada
Pendiente: se completa después de la exposición.				

Preguntas formuladas para otros equipos

Preguntas críticas preparadas por el equipo para la ronda de preguntas. Cuestionan una decisión de diseño, no piden repetir la explicación. Durante la clase se anota a qué equipo se dirigió cada una y quién la formuló.

N.º	Pregunta	Qué cuestiona	Equipo / quién pregunta
1	Si su servicio confirma la operación pero la respuesta se pierde, ¿cómo evitan el doble efecto cuando el cliente reintenta?	Idempotencia y consistencia ante fallos de red	
2	¿Por qué eligieron mensajería en esa interacción? ¿Qué pasa si la cola se llena?	Elección del mecanismo, volumen de mensajes y tolerancia a fallos	
3	¿Dónde está su punto único de fallo y qué pasa si se cae?	Disponibilidad y tolerancia a fallos	
4	¿Cómo garantizan el orden de los eventos de un mismo usuario?	Orden de los mensajes asíncronos y consistencia	
5	¿Qué pasa con la operación si la base de datos no responde?	Consistencia y manejo de fallos en la persistencia	